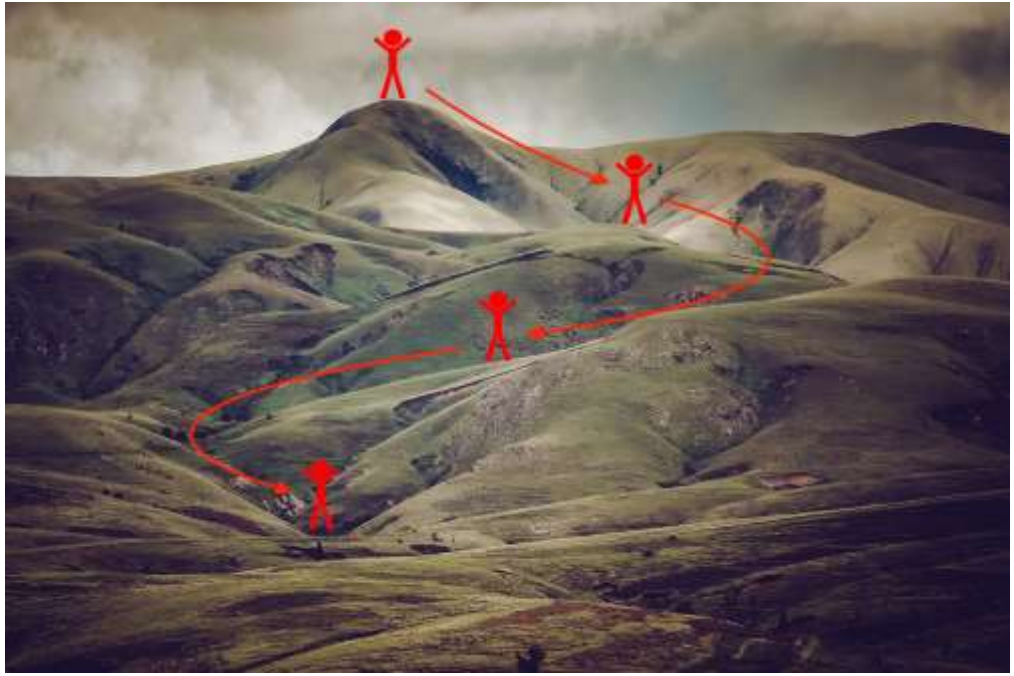
FPT SCHOOL OF BUSINESS & TECHNOLOGY

AI - MACHINE LEARNING

PhD. Truc Thi Kim, Nguyen

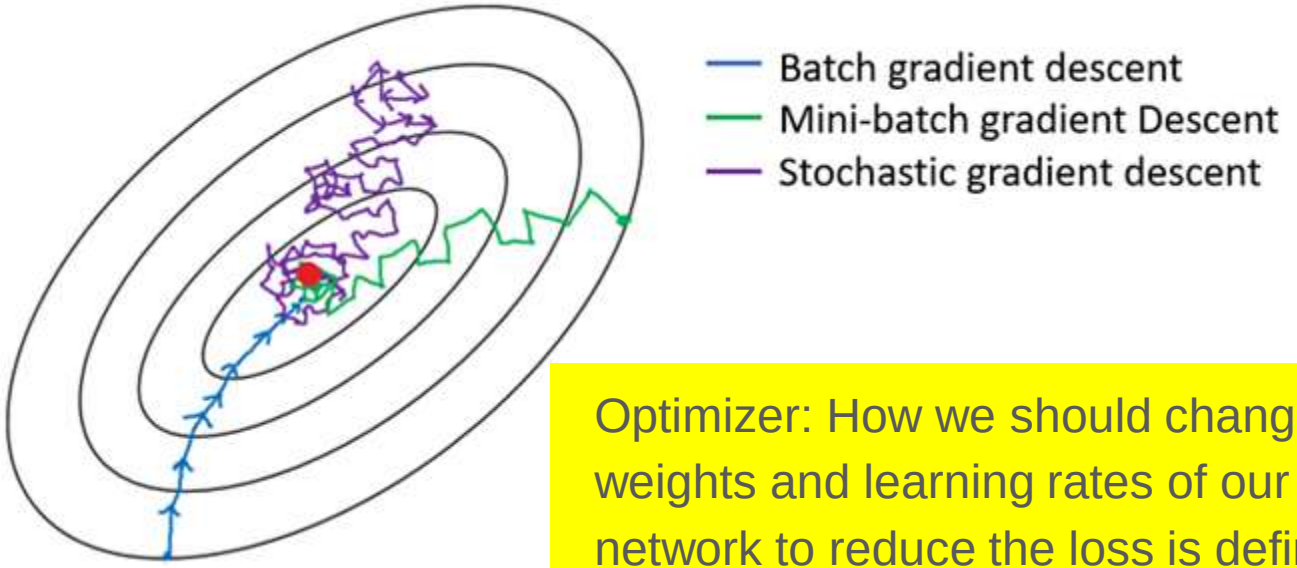
093 293 9981 info@fsb.edu.vn https://fsb.edu.vn

Optimization algorithms



FSB

Optimization algorithms



Optimizer: How we should change the weights and learning rates of our neural network to reduce the loss is defined by the optimizer we use.

FSB

Mini-batch gradient descent

- Training NN with a large data is slow. So to find an optimization algorithm that runs faster is a good idea.
- Suppose we have m = 50 million. To train this data it will take a huge processing time for one step. because 50 million won't fit in the memory at once we need other processing to make such a thing.
- It turns out you can make a faster algorithm to make gradient descent process some of your items even before you finish the 50 million items.
- Suppose we have split m to mini batches of size 1000.
  - X{1} = 0 ... 1000
  - X{2} = 1001 ... 2000
  - ...
  - X{bs} = ...
- We similarly split X & Y.

- So the definition of mini batches ==> t: X{t}, Y{t}

FSB

Mini-Batch gradient descent

- In **Batch gradient descent** we run the gradient descent on the whole dataset.
- While in **Mini-Batch gradient descent** we run the gradient descent on the mini datasets.
- Mini-Batch algorithm pseudo code:

```
for t = 1:No_of_batches                          # this is called an epoch
        AL, caches = forward_prop(X{t}, Y{t})
        cost = compute_cost(AL, Y{t})
        grads = backward_prop(AL, caches)
        update_parameters(grads)
```

- The code inside an epoch should be vectorized.
- Mini-batch gradient descent works much faster in the large datasets.

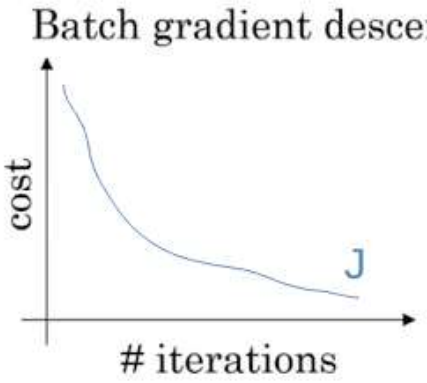
FSB

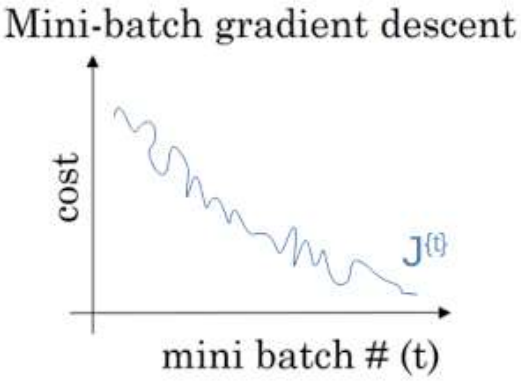
Mini-Batch gradient descent

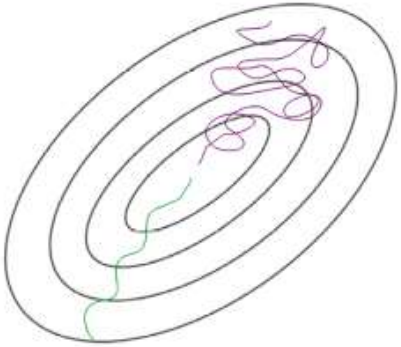
- Guidelines for choosing mini-batch size:
  - If small training set (< 2000 examples) - use batch gradient descent.
  - It has to be a power of 2 (because of the way computer memory is layed out and accessed, sometimes your code runs faster if your mini-batch size is a power of 2): 64, 128, 256, 512, 1024, ...
  - Make sure that mini-batch fits in CPU/GPU memory.
- Mini-batch size is a hyperparameter.

FSB

Mini-batch gradient descent







If mini-batch size = 1: Stocastic Gradient Descent = every example (row) it is used as mini-batch: lose sped-up from vectorization.

If mini-batch size = m: Batch Gradient Descent: $(X^{\{1\}}, Y^{\{1\}}) = (X, Y)$ = the entire training set is used: size = n: too long interations.

FSB

Mini-batch gradient descent

- In mini-batch algorithm, the cost won't go down with each step as it does in batch algorithm. It could contain some ups and downs but generally it has to go down (unlike the batch gradient descent where cost function decreases on each iteration).
- Mini-batch size:
  - (mini batch size = m) ==> Batch gradient descent
  - (mini batch size = 1) ==> Stochastic gradient descent (SGD)
  - (mini batch size = between 1 and m) ==> Mini-batch gradient descent
- Batch gradient descent:
  - too long per iteration (epoch)
- Stochastic gradient descent:
  - too noisy regarding cost minimization (can be reduced by using smaller learning rate)
  - won't ever converge (reach the minimum cost)
  - lose speedup from vectorization
- Mini-batch gradient descent:
  - faster learning: you have the vectorization advantage make progress without waiting to process the entire training set.
  - doesn't always exactly converge (oscillates in a very small region, but you can reduce learning rate)

FSB

Temperature in London

$\theta_1 = 40°F$ 4°C

$\theta_2 = 49°F$ 9°C

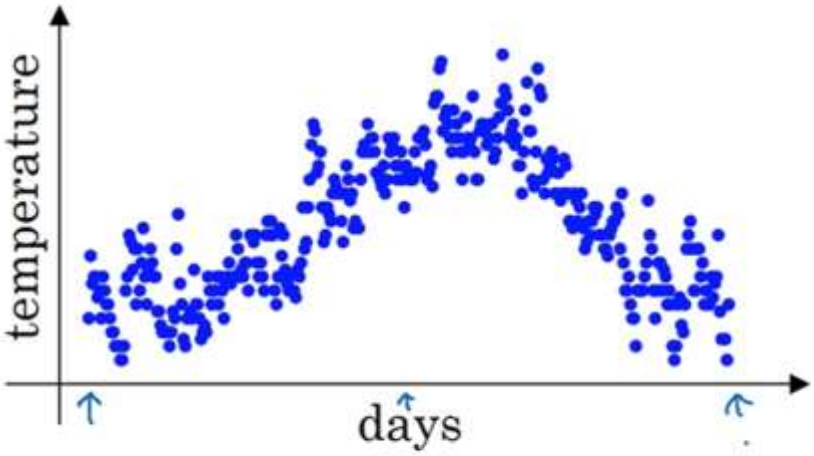
$\theta_3 = 45°F$

$\vdots$

$\theta_{180} = 60°F$ 15°C

$\theta_{181} = 56°F$

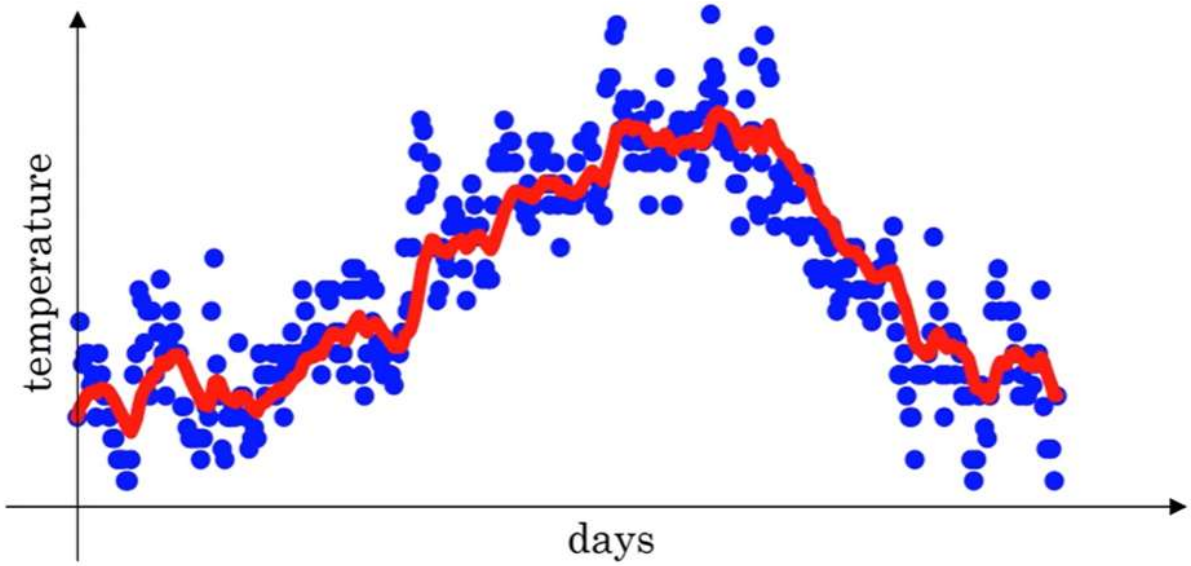
$\vdots$



Exponentially weighted averages

$$v_t = \beta v_{t-1} + (1-\beta)\theta_t$$

$\beta = 0.9$ 0.98

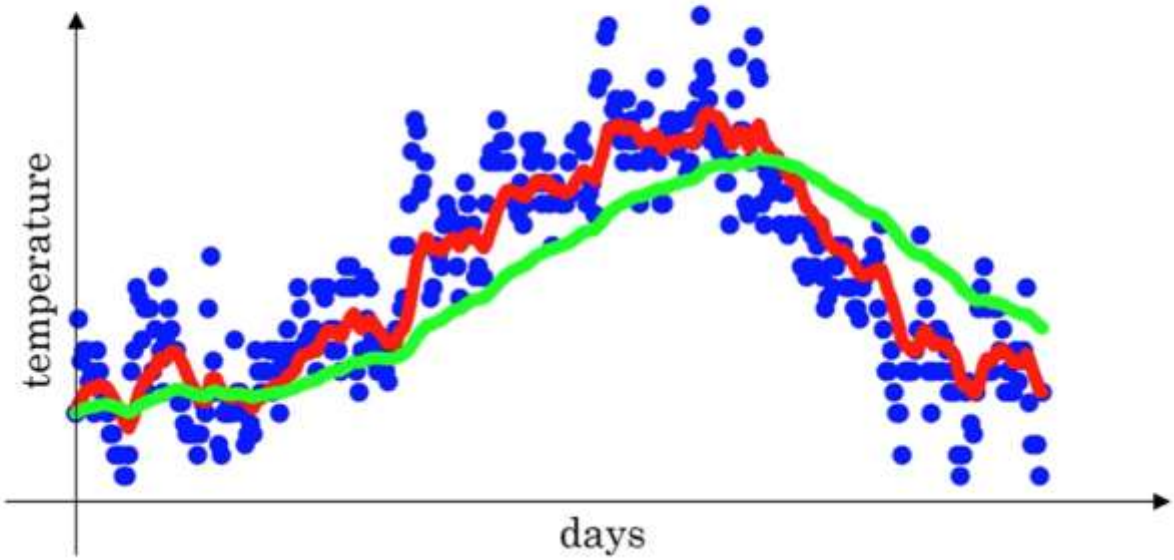


FSB

Exponentially weighted averages

$$v_t = \beta v_{t-1} + (1-\beta)\theta_t$$

$\beta = 0.9$ 0.98

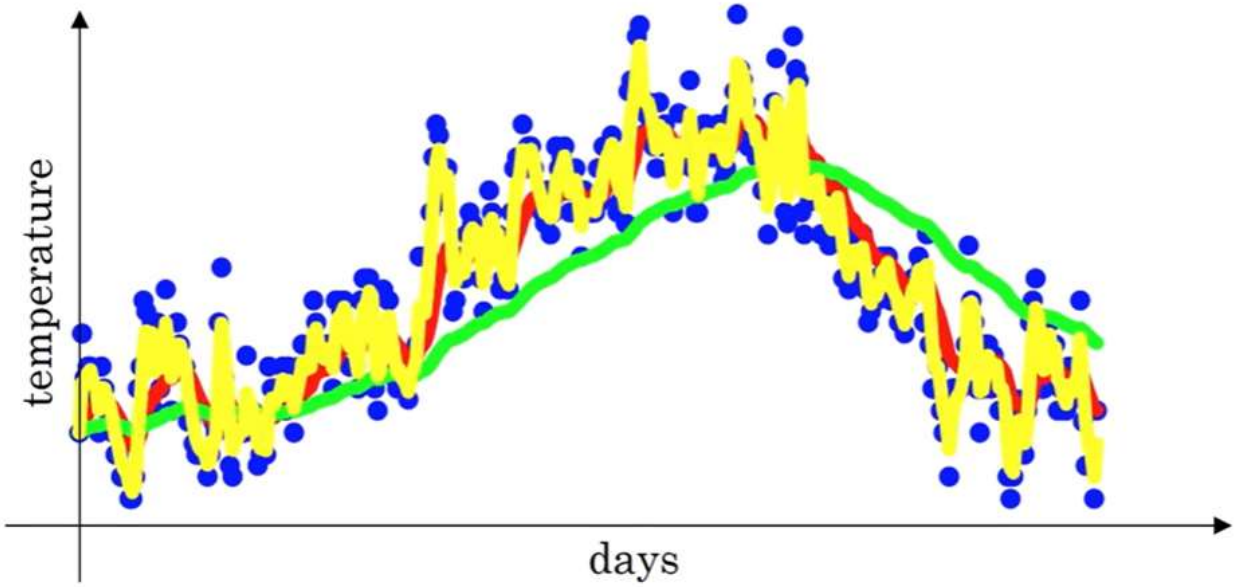


FSB

Exponentially weighted averages

$$v_t = \beta v_{t-1} + (1-\beta)\theta_t$$

$\beta = 0.9$ 0.98 0.5



FSB

Implementing exponentially weighted averages

$$v_0 = 0$$
$$v_1 = \beta v_0 + (1-\beta)\,\theta_1$$
$$v_2 = \beta v_1 + (1-\beta)\,\theta_2$$
$$v_3 = \beta v_2 + (1-\beta)\,\theta_3$$
$$\ldots$$

$V_\theta := 0$

$V_\theta := \beta v + (1-\beta)\theta_1$

$V_\theta := \beta v + (1-\beta)\theta_2$

$\vdots$

```
v = 0
Repeat
{
        Get theta(t)
        v = beta * v + (1-beta) * theta(t)
}
```

$V_\theta = 0$

Repeat {

Get next θ_t

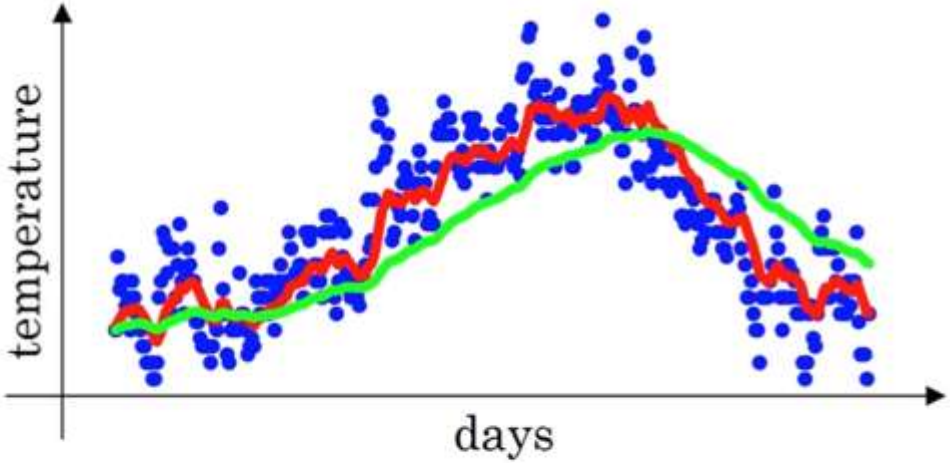
$V_\theta := \beta V_\theta + (1-\beta)\theta_t$

}

Andrew Ng

FSB

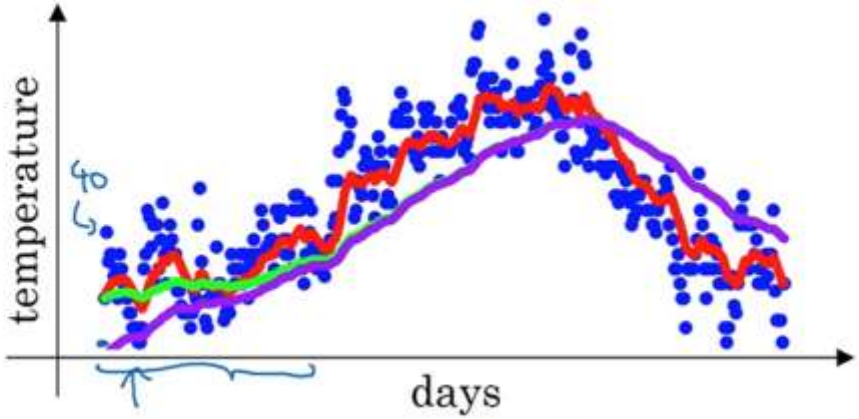
Bias correction



→ $v_t = \beta v_{t-1} + (1-\beta)\theta_t$

FSB

Bias correction



$\beta = 0.98$

$\rightarrow v_t = \beta v_{t-1} + (1-\beta)\theta_t$

$V_0 = 0$

$V_1 = \cancel{0.98 V_0 +} \; 0.02\,\theta_1$

$V_2 = 0.98\,V_1 + 0.02\,\theta_2$

$= 0.98 \times 0.02 \times \theta_1 + 0.02\,\theta_2$

$= 0.0196\,\theta_1 + 0.02\,\theta_2$

$\dfrac{V_t}{1-\beta^t}$

$t=2: \quad 1-\beta^t = 1-(0.98)^2 = 0.0396$

$\dfrac{V_2}{0.0396} = \dfrac{0.0196\,\theta_1 + 0.02\,\theta_2}{0.0396}$

Andrew Ng

Bias correction in exponentially weighted averages

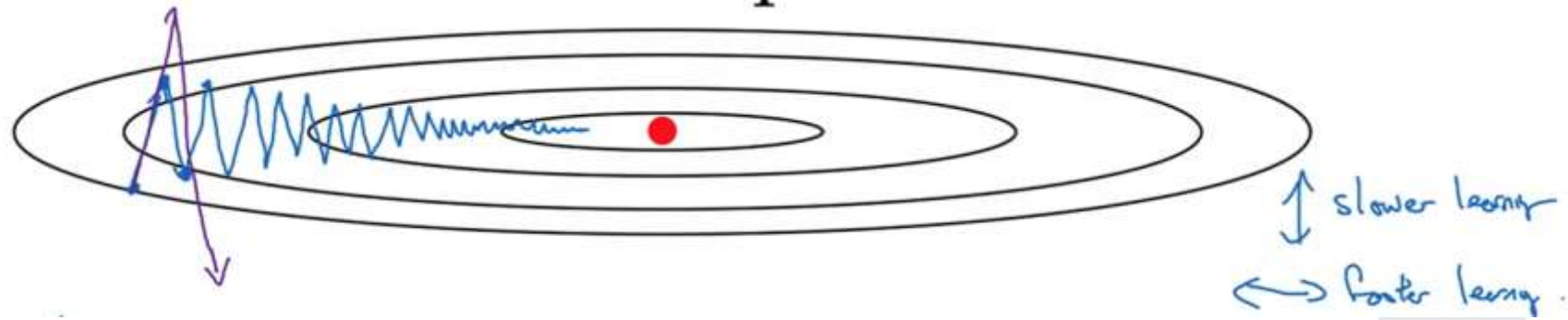
- The bias correction helps make the exponentially weighted averages more accurate.
- Because `v(0) = 0`, the bias of the weighted averages is shifted and the accuracy suffers at the start.
- To solve the bias issue we have to use this equation:

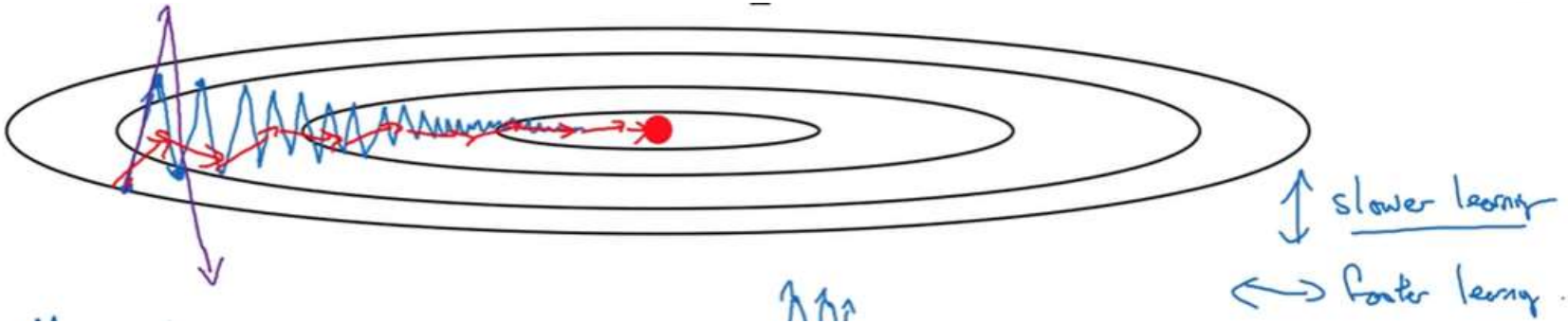
```
v(t) = (beta * v(t-1) + (1-beta) * theta(t)) / (1 - beta^t)
```

- As t becomes larger the `(1 - beta^t)` becomes close to `1`

Gradient descent with momentum

Gradient descent example





Gradient descent with momentum

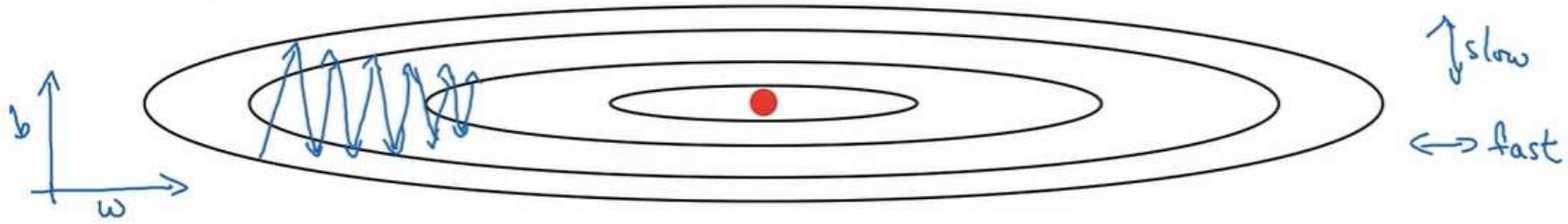
- The momentum algorithm almost always works faster than standard gradient descent.
- The simple idea is to calculate the exponentially weighted averages for your gradients and then update your weights with the new values.
- Pseudo code:

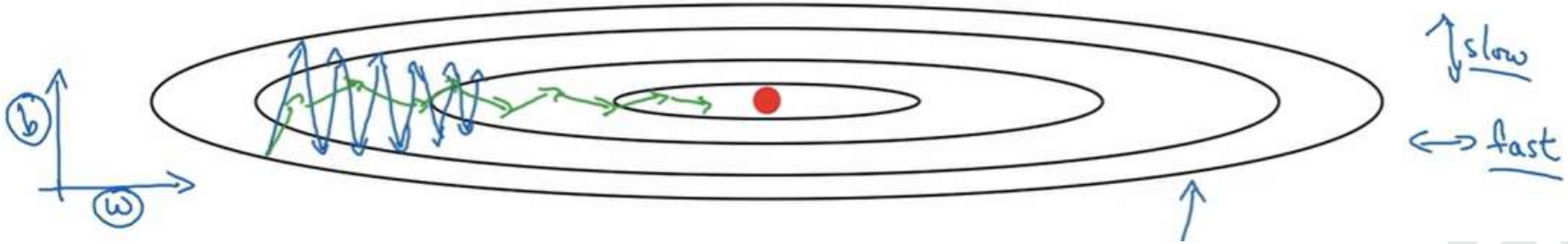
Momentum helps the cost function to go to the minimum point in a more fast and consistent way.
beta is another hyperparameter. beta = 0.9 is very common and works very well in most cases.
In practice people don't bother implementing bias correction.

```
vdW = 0, vdb = 0
on iteration t:
        # can be mini-batch or batch gradient descent
        compute dw, db on current mini-batch

        vdW = beta * vdW + (1 - beta) * dW
        vdb = beta * vdb + (1 - beta) * db
        W = W - learning_rate * vdW
        b = b - learning_rate * vdb
```

RMSprop





RMSprop

- Stands for Root mean square prop.
- This algorithm speeds up the gradient descent.
- Pseudo code:

```
sdW = 0, sdb = 0
on iteration t:
        # can be mini-batch or batch gradient descent
        compute dw, db on current mini-batch

        sdW = (beta * sdW) + (1 - beta) * dW^2  # squaring is element-wise
        sdb = (beta * sdb) + (1 - beta) * db^2  # squaring is element-wise
        W = W - learning_rate * dW / sqrt(sdW)
        b = B - learning_rate * db / sqrt(sdb)
```

RMSprop will make the cost function move slower on the vertical direction and faster on the horizontal direction in the following example:
Ensure that sdW is not zero by adding a small value epsilon (e.g. epsilon = 10^-8) to it:
W = W - learning_rate * dW / (sqrt(sdW) + epsilon)
With RMSprop you can increase your learning rate.

Geoffrey Hinton "godfather of AI", or "godfather of deep learning",

- His work is primarily focused on machine learning algorithms, and he has aimed both to create tools that allow computers to find structures in complex data, as well as showing that is how the human brain learns. He is most famous for pioneering the back-propagation algorithm that helps train neural networks, but he and his students have helped build an array of technologies that underpin today's artificial intelligence.

Geoffrey Hinton: Who is the 'godfather of AI', whose warning about artificial intelligence has shaken the world? | The Independent

Adam optimization algorithm

- Stands for Adaptive Moment Estimation.
- Adam optimization and RMSprop are among the optimization algorithms that worked very well with a lot of NN architectures.
- Adam optimization simply puts RMSprop and momentum together!
- Pseudo code:

```
vdW = 0, vdW = 0
sdW = 0, sdb = 0
on iteration t:
        # can be mini-batch or batch gradient descent
        compute dw, db on current mini-batch

        vdW = (beta1 * vdW) + (1 - beta1) * dW     # momentum
        vdb = (beta1 * vdb) + (1 - beta1) * db     # momentum

        sdW = (beta2 * sdW) + (1 - beta2) * dW^2   # RMSprop
        sdb = (beta2 * sdb) + (1 - beta2) * db^2   # RMSprop

        vdW = vdW / (1 - beta1^t)      # fixing bias
        vdb = vdb / (1 - beta1^t)      # fixing bias

        sdW = sdW / (1 - beta2^t)      # fixing bias
        sdb = sdb / (1 - beta2^t)      # fixing bias

        W = W - learning_rate * vdW / (sqrt(sdW) + epsilon)
        b = B - learning_rate * vdb / (sqrt(sdb) + epsilon)
```

Adam optimization algorithm

- Hyperparameters for Adam:
  - Learning rate: needed to be tuned.
  - beta1: parameter of the momentum - 0.9 is recommended by default.
  - beta2: parameter of the RMSprop - 0.999 is recommended by default.
  - epsilon: 10^-8 is recommended by default.

FSB

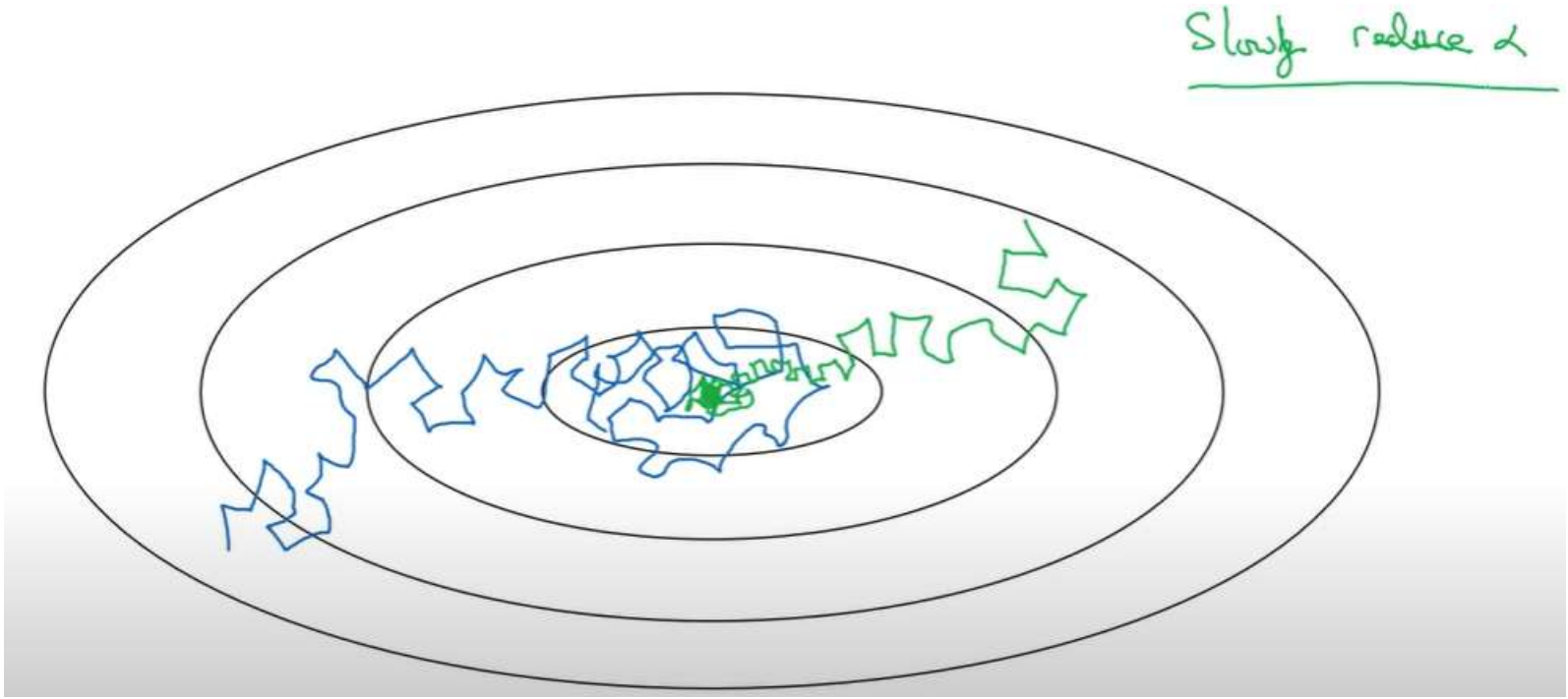
Adam Coates – Author of Adam

I am currently a Director at Apple.

I received my PhD from Stanford University in 2012 and was the director of the Silicon Valley AI Lab at **Baidu Research** until September 2017, then an Operating Partner at **Khosla Ventures** until 2018. During my graduate career, I co-developed an **autonomous aerobatic helicopter**, worked on perception systems for household robots, and early large-scale deep learning methods. I developed deep learning software for high-performance computing systems with a team at Stanford, used for unsupervised learning, object detection and self-driving cars.

FSB

Learning rate decay



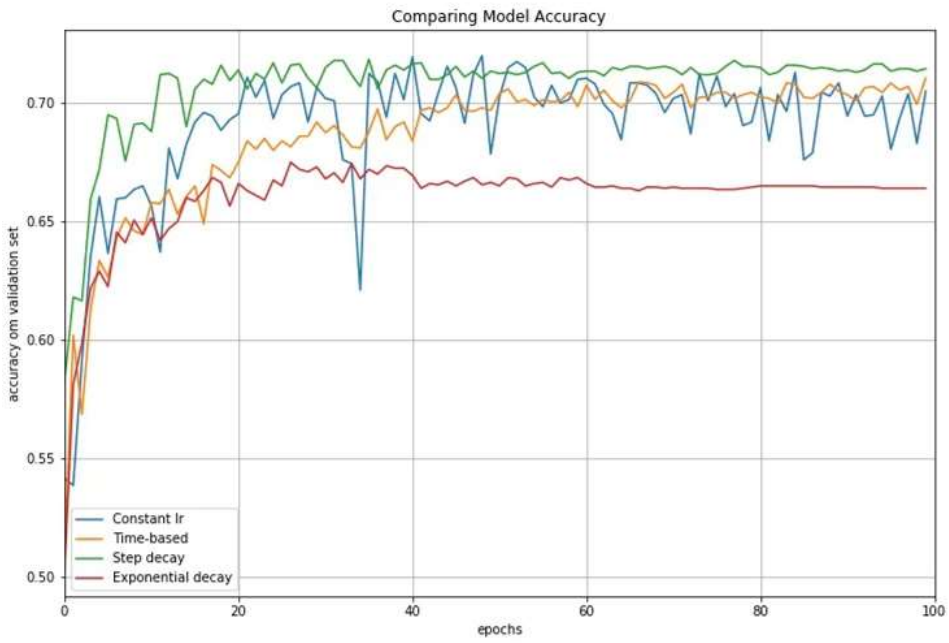
FSB

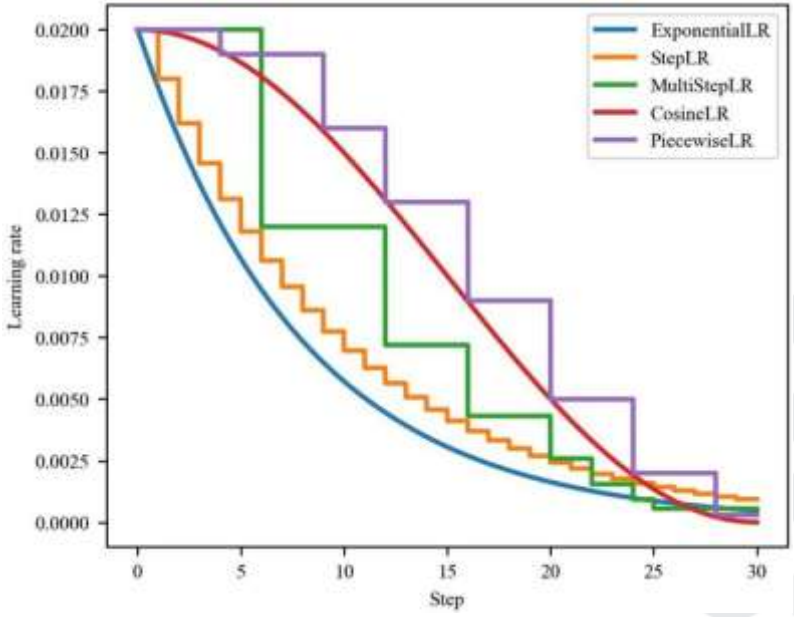
Learning rate decay

- Slowly reduce learning rate.
- As mentioned before mini-batch gradient descent won't reach the optimum point (converge). But by making the learning rate decay with iterations it will be much closer to it because the steps (and possible oscillations) near the optimum are smaller.
  - One technique equations islearning_rate = (1 / (1 + decay_rate * epoch_num)) * learning_rate_0
- epoch_num is over all data (not a single mini-batch).
- Other learning rate decay methods (continuous):
  - learning_rate = (0.95 ^ epoch_num) * learning_rate_0
  - learning_rate = (k / sqrt(epoch_num)) * learning_rate_0
- Some people perform learning rate decay discretely - repeatedly decrease after some number of epochs.
- Some people are making changes to the learning rate manually.
- decay_rate is another hyperparameter.

FSB

Other learning rate decays



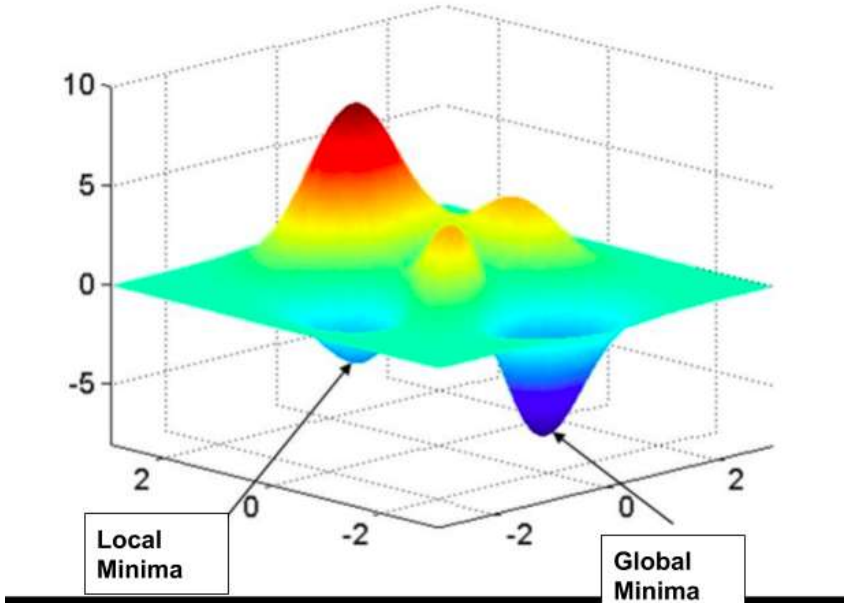
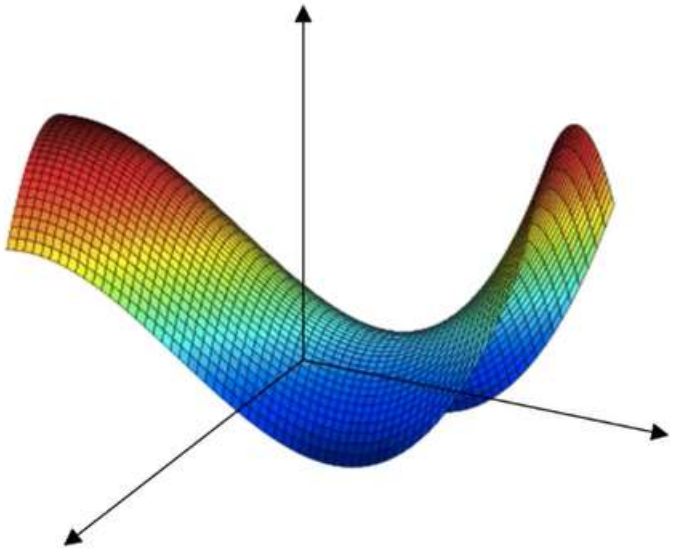


https://www.geeksforgeeks.org/learning-rate-decay/

https://medium.com/towards-data-science/learning-rate-schedules-and-adaptive-learning-rate-methods-for-deep-learning-2c8f433990d1

The problem of local optima

- The normal local optima is not likely to appear in a deep neural network because data is usually high dimensional. For point to be a local optima it has to be a local optima for each of the dimensions which is highly unlikely.
- It's unlikely to get stuck in a bad local optima in high dimensions, it is much more likely to get to the saddle point rather to the local optima, which is not a problem.
- Plateaus can make learning slow:
- Plateau is a region where the derivative is close to zero for a long time.
- This is where algorithms like momentum, RMSprop or Adam can help.

FSB

The problem of local optima